Fixing inconsistencies between `constexpr` and `consteval` functions

Document Number: P1937R1
Date: 2020-01-12
Author: David Stone (david.stone@uber.com, david@doublewise.net)
Audience: Core Working Group (CWG)

Summary

A `consteval` function is the same as a `constexpr` function except that:

1. A `consteval` function is guaranteed to be evaluated at compile time.
2. You cannot have a pointer to a `consteval` function except in the body of another `consteval` function.
3. All overrides of a virtual function must be `consteval` if the base function is `consteval`, otherwise they must not be `consteval`.
4. A `consteval` function is evaluated even in contexts that would otherwise be unevaluated.

The first point is the intended difference between the two functions. The second point is a consequence of the first. The third point (`virtual` functions) ends up making sense as a difference between them because of what is fundamentally possible. The final point seems like an unfortunate inconsistency in the language. This paper proposes changing the rules surrounding bullet 4 to unify the behavior of `consteval` and `constexpr` functions for C++20.

Revision history

Since R0

- Removed section on virtual functions: it will be its own paper in the future, targeted at C++23.
- Added a small piece of wording

Unevaluated contexts

Requiring evaluation of `consteval` functions in unevaluated contexts does not seem necessary. When I asked about this discrepancy on the core reflector, the answer that I got was essentially that evaluation of a `consteval` function may have side-effects on the abstract machine, and we would

like to preserve these side-effects. However, `consteval` functions are not special in this ability, so it seems strange to have a rule that makes them special. It seems to me that we should either

1. not have a special case for `consteval`, which would mean that it is not evaluated in an unevaluated operand, or
2. Rethink the concept of "unevaluated operands".

This paper argues in favor of option 1, as option 2 is a breaking change that doesn't seem to bring much benefit.

My understanding is that the status quo means the following is valid:

```
constexpr auto add1(auto lhs, auto rhs) {
    return lhs + rhs;
}
using T = decltype(add1(std::declval<int>(), std::declval<int>()));
```

but this very similar code is ill-formed

```
consteval auto add2(auto lhs, auto rhs) {
    return lhs + rhs;
}
using T = decltype(add2(std::declval<int>(), std::declval<int>()));
```

Wording

Note: this wording is presumed incomplete. There is a note stating that `consteval` functions are always evaluated, but I cannot find where the normative wording actually states this.

Modify [expr.const], paragraph 13, as follows:

> ~~[Note: A manifestly constant-evaluated expression is evaluated even in an unevaluated operand. — end note]~~